Повышение квалификации специалистов по информационной безопасности

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
ДЛЯ СТУДЕНТА 2

по курсу
«Методы и принципы проектирования современных защищённых систем. Применение механизмов искусственного интеллекта для анализа данных при построении систем информационной безопасности»

SECURITY GATE РЕЛИЗ-КАНДИДАТА

SAST, секреты, SBOM, зависимости и контейнер

(65 минут)

2026

СОДЕРЖАНИЕ

Сценарий и результат 3

0. Подготовка — 5 минут 4

1. Обычная сборка — 5 минут 5

2. SAST: двойное освобождение — 12 минут 6

    2.1. Отчёт в формате SARIF 7

3. DAST: динамический анализ с ASAN — 3 минуты 9

4. Поиск секретов — 10 минут 11

5. Образ, SBOM и зависимости — 12 минут 12

6. Контейнер — 9 минут 13

7. Исправление — 6 минут 14

    7.1. Единая точка освобождения 14

    7.2. Проверка индекса 14

    7.3. Контейнер 15

    7.4. Секреты и история Git 15

8. Повторный Security Gate — 3 минуты 17

Сценарий и результат

Команда готовит к выпуску `release-agent 2.0` — небольшую программу на C в Docker-образе. Ваша роль — участник Security Gate. Каждый этап оставляет машиночитаемый отчёт в `/work/reports/`; выпуск разрешается только после повторной проверки.

После единственного входа в Ubuntu-контейнер вы работаете в `/work/playbook.sh`: переносите команды из методички в функцию соответствующего этапа вместо `:`. Исходный код находится в `/work/repository`, отчёты — в `/work/reports`.

Таблица 1 — Маршрут практики — 65 минут

Время	Контроль	Артефакт
5 мин	Подготовка	рабочий Git-репозиторий
5 мин	Обычная сборка	тест пройден
12 мин	Clang Static Analyzer	`sast.txt`, `sast.sarif`
3 мин	AddressSanitizer	`dast-asan.txt`
10 мин	Gitleaks	`gitleaks.json`
12 мин	Syft и Grype	`sbom.cdx.json`, `grype.json`
9 мин	Trivy	`trivy.json`
6 мин	Исправление	исправленный репозиторий
3 мин	Повторный Gate	решение `PASS` или `BLOCK`

0. Подготовка — 5 минут

Откройте терминал и выполните:

```
./lab.sh start
./lab.sh shell
```

Команды выполняются из выданного каталога `materials`. Первая команда один раз собирает единый образ на Ubuntu 24.04 и запускает его. Вторая открывает Bash внутри контейнера. При первом старте автоматически создаются `/work/repository` с двумя коммитами, каталог отчётов и playbook.

Все дальнейшие команды выполняйте в открытом контейнере. Проверьте стенд:

```
pwd
tree -a -L 2 /work
clang --version | head -1
gitleaks version
syft version
grype version
trivy --version | head -1
```

Рабочий путь — `/work`. Откройте шаблон командой `nano /work/playbook.sh`. В `/work/repository` ожидаются два коммита и файлы `src/session.c`, `Dockerfile`, `compose.yaml`, `config/app.env`.

1. Обычная сборка — 5 минут

В функции `baseline()` замените `:` следующим кодом:

```
cmake -S . -B build -DCMAKE_C_COMPILER=clang
cmake --build build
ctest --test-dir build
./build/release-agent
```

Запустите:

```
./playbook.sh baseline
```

Сборка завершается успешно, тест проходит, а штатный запуск печатает `session closed`. На этом этапе признаков ошибки нет. Однако тест проверяет только один маршрут, поэтому решение о безопасности релиза пока принимать рано.

2. SAST: двойное освобождение — 12 минут

В `scan_sast()` запишите:

```
local output
if ! output="$(clang --analyze -Xanalyzer -analyzer-checker=unix.Malloc \
  -Xanalyzer -analyzer-output=text -Iinclude src/session.c 2>&1)"; then
  printf "%s\n" "$output" | tee /work/reports/sast.txt
  echo "SAST остановлен: src/session.c не компилируется" >&2
  return 1
fi
if [[ -n "$output" ]]; then
  printf "%s\n" "$output" | tee /work/reports/sast.txt
else
  echo "No analyzer findings" | tee /work/reports/sast.txt
fi
clang --analyze -Xanalyzer -analyzer-checker=unix.Malloc \
  -Xanalyzer -analyzer-output=sarif -Iinclude src/session.c \
  -o /work/reports/sast.sarif
```

Первый блок `if` нужен на случай, когда файл не компилируется (например, после неаккуратной правки): тогда на экран выводятся ошибки компилятора и этап останавливается с понятным сообщением, а не завершается молча. Запустите `./playbook.sh scan_sast`. Прочитайте трассу снизу вверх и найдите три события: выделение памяти в строке 8, первое освобождение в строке 23 и повторное освобождение в строке 27.

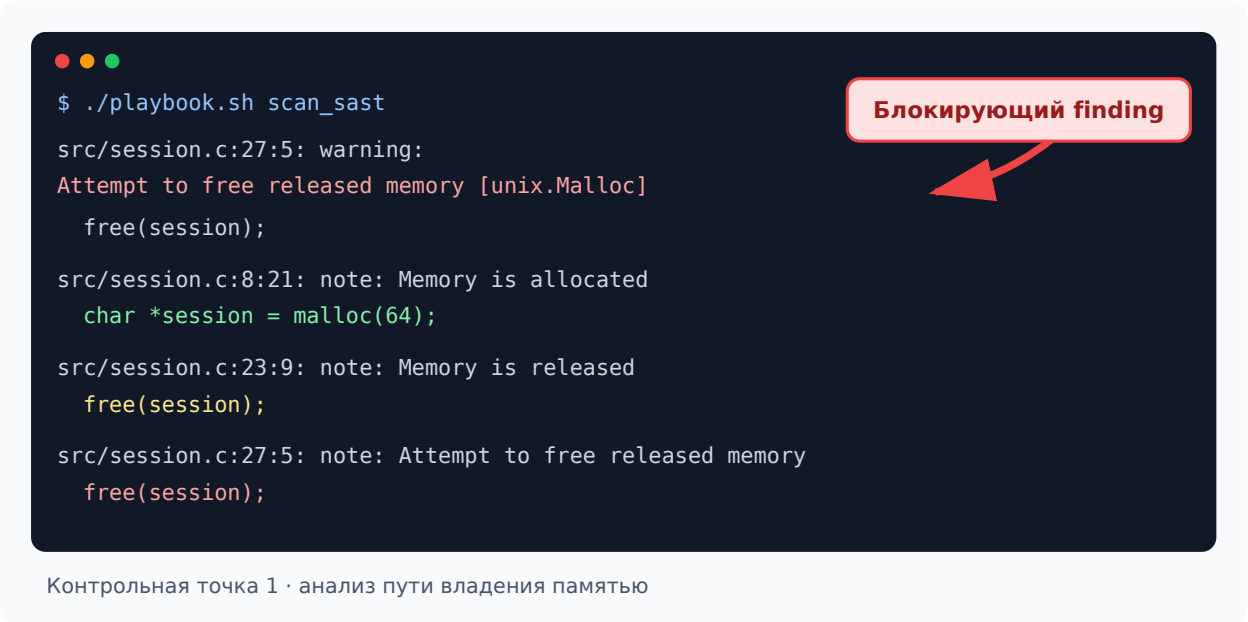


Рисунок 1 — Clang строит путь до ошибки, а не просто совпадение текста

2.1. Отчёт в формате SARIF

Текстовая трасса удобна для чтения, но её трудно обработать программой. Поэтому вторая команда в `scan_sast()` пишет тот же результат в файл `/work/reports/sast.sarif` в формате SARIF (Static Analysis Results Interchange Format, версия 2.1.0, стандарт OASIS). Это JSON, который понимают GitHub Code Scanning, GitLab, Azure DevOps и другие инструменты: отчёты разных анализаторов можно собирать в одном месте. Файл создаёт `clang` с параметром `-analyzer-output=sarif`, а путь к нему задаёт ключ `-o`. Сам файл не печатается на экран, поэтому откройте его с помощью `jq`:

```
jq '.version, .runs[0].tool.driver.name' /work/reports/sast.sarif
jq -r '.runs[0].results[]
  | [.ruleId, .level, .locations[0].physicalLocation.region.startLine,
     .message.text] | @tsv' /work/reports/sast.sarif
jq -r '.runs[0].results[0].codeFlows[0].threadFlows[0].locations[]
  | [.importance, .location.physicalLocation.region.startLine,
     .location.message.text] | @tsv' /work/reports/sast.sarif
```

Первая команда печатает версию формата и имя анализатора (`"2.1.0"` и `"clang"`). Вторая показывает находку одной строкой через табуляции: правило, важность, строка и сообщение (`unix.Malloc`, `warning`, `27`, `Attempt to free released memory`). Третья выводит шаги трассы с их важностью и номерами строк. В них вы увидите те же три события: «Memory is allocated» (строка 8), «Memory is released» (строка 23) и «Attempt to free released memory» (строка 27).

Структура отчёта вложена так: `runs` → `tool` и `results` → `codeFlows`. Основные поля описаны в таблице.

Таблица 2 — Основные поля SARIF-отчёта Clang

Поле	Назначение
`version`, `$schema`	Версия формата (`2.1.0`) и адрес схемы, по которой отчёт можно проверить
`runs`	Список запусков анализатора. Один файл может объединять несколько инструментов; у нас один запуск
`runs[].tool.driver`	Анализатор: `name` (`clang`), `version` (версия Clang), `informationUri` (документация)
`tool.driver.rules[]`	Описание правил (проверок). `id` и `name` — `unix.Malloc`, `fullDescription.text` — что проверяет правило, `helpUri` — справка, `defaultConfiguration.level` — важность по умолчанию
`runs[].artifacts[]`	Файлы, на которые ссылается отчёт: `location.uri` (путь), `length` (размер), `roles` (`resultFile` — файл с находкой)

Поле	Назначение
`runs[].results[]`	Список находок. У нас одна — двойное освобождение
`results[].ruleId`, `ruleIndex`	Какое правило сработало: идентификатор и номер в списке `rules`
`results[].level`	Важность находки: `error`, `warning`, `note` или `none`. Для `unix.Malloc` это `warning`
`results[].message.text`	Сообщение для человека: `Attempt to free released memory`
`results[].locations[]`	Место ошибки. `physicalLocation.artifactLocation.uri` — файл, `region` — `startLine`, `startColumn`, `endLine`, `endColumn` (у нас строка 27, столбцы 5–18)
`results[].codeFlows[]`	Путь выполнения программы до ошибки. В `threadFlows[].locations[]` лежат шаги в порядке выполнения
`locations[].importance`	Значимость шага: `essential` — ключевое событие, `important` — важное условие, `unimportant` — пояснение (например, «Taking true branch»)
`locations[].location.message.text`	Что происходит на шаге: выделение памяти, ветвление, освобождение

Номера строк и столбцов в SARIF отсчитываются с единицы, а `uri` в учебной среде содержит абсолютный путь к файлу внутри контейнера. Если находок нет, массив `results` пуст, но файл всё равно создаётся: по нему видно, что анализатор запускался.

Ответьте до исправления: почему присваивание `session = NULL` после первого `free()` скроет падение, но оставит неясное владение ресурсом? Правильная цель — сделать одну точку освобождения.

3. DAST: динамический анализ с ASAN — 3 минуты

SAST читает код, но не запускает его. Динамический анализ (в практике SDL его называют DAST, динамическим тестированием) делает наоборот: программу собирают с проверками и запускают на реальных входных данных. Здесь вместо сетевого сканера веб-приложений мы применяем AddressSanitizer (ASAN) из компилятора. Он добавляет в программу проверки памяти и останавливает её сразу, когда она выходит за границы массива, использует освобождённую память или освобождает её дважды. Тот же механизм ищет утечки: при завершении программы LeakSanitizer (LSan, входит в ASAN) печатает память, которую не освободили.

В `scan_dast()` запишите:

```
clang -fsanitize=address -fno-omit-frame-pointer -g -Iinclude \
  src/main.c src/session.c -o /tmp/release-agent-asan
{
  echo "== run without arguments"
  /tmp/release-agent-asan || true
  echo "== run with --slot 2"
  /tmp/release-agent-asan --slot 2 || true
  echo "== run with --slot 9"
  /tmp/release-agent-asan --slot 9 || true
} 2>&1 | tee /work/reports/dast-asan.txt
```

Сборка идёт тем же компилятором `clang`, что и SAST; ключ `-fsanitize=address` подключает ASAN. Параметр `-g` нужен, чтобы в отчёте были имена функций и номера строк (имена подставляет `llvm-symbolizer`, он есть в образе). Программа запускается три раза: без аргументов (штатный сценарий), с корректным номером квоты `--slot 2` и с номером за пределами таблицы `--slot 9`. Часть `|| true` не даёт playbook остановиться, когда ASAN завершает программу с ошибкой.

Запустите `./playbook.sh scan_dast`. Первые два запуска завершаются без замечаний (`session closed` и `quota: 30`). Третий запуск печатает отчёт ASAN. Найдите в нём три места:

- первая строка: `ERROR: AddressSanitizer: global-buffer-overflow`, вид ошибки;
- в стеке `#0 ... in session_quota /work/repository/src/session.c:34:12` (функция, файл, строка и столбец, где программа читает за границей массива) и `#1 ... in main`, откуда эта функция вызвана;

– в конце строка `SUMMARY: AddressSanitizer: global-buffer-overflow ... in session_quota`, итог для отчёта.

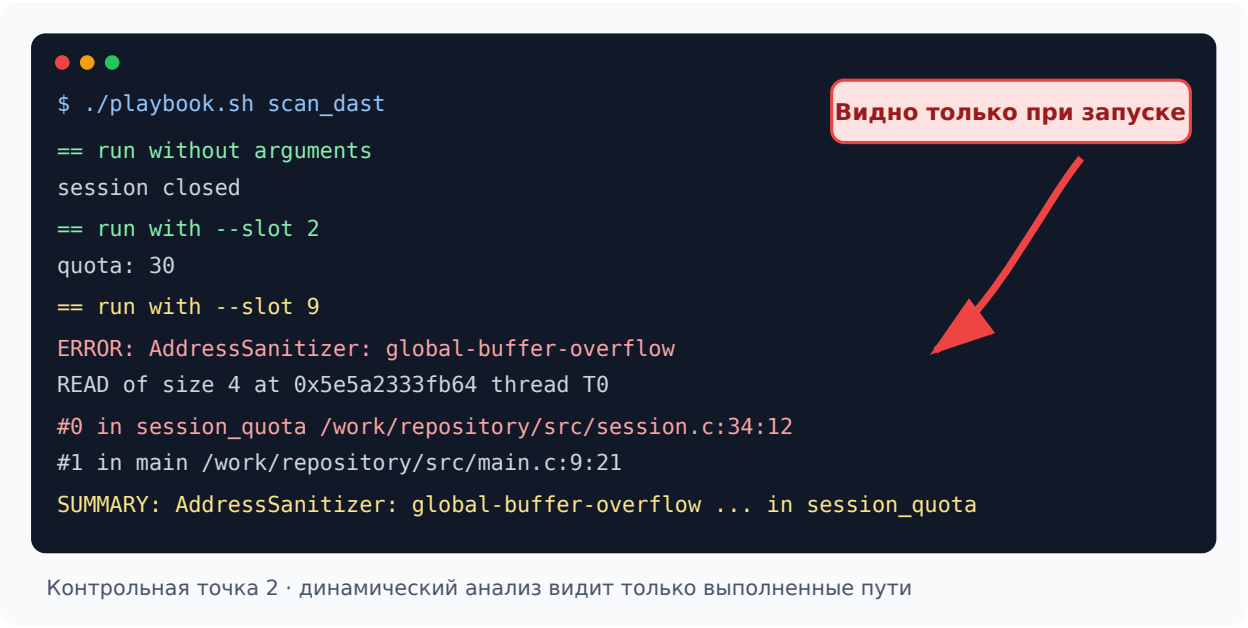


Рисунок 2 — Ошибка видна только на запуске с неверным номером квоты

Это чтение элемента `quota[9]` в таблице из четырёх значений. Номер приходит из командной строки, и SAST его значения не знает: поэтому Clang в разделе 2 эту ошибку не нашёл. Обратное тоже верно: двойное освобождение из раздела 2 здесь не появилось, потому что мы не запускали программу с `--reject`. Динамический анализ видит только те пути, которые выполнены.

Ответьте до исправления: что показал бы ASAN, если запустить `/tmp/release-agent-asan --reject`, и почему одного DAST для приёмки мало?

4. Поиск секретов — 10 минут

В `scan_secrets()` запишите:

```
gitleaks detect --source . --config /opt/practice/gitleaks.toml \
  --log-opts=--all --report-format json \
  --report-path /work/reports/gitleaks.json --no-banner || true
```

Запустите:

```
./playbook.sh scan_secrets
```

Откройте `reports/gitleaks.json`. Должны быть два finding: токен в текущем `config/app.env` и токен из удалённого `config/legacy.env`. Ключ `--log-opts=--all` заставляет Gitleaks проверить всю историю, а `|| true` не прерывает учебный playbook при ожидаемом блокирующем результате.

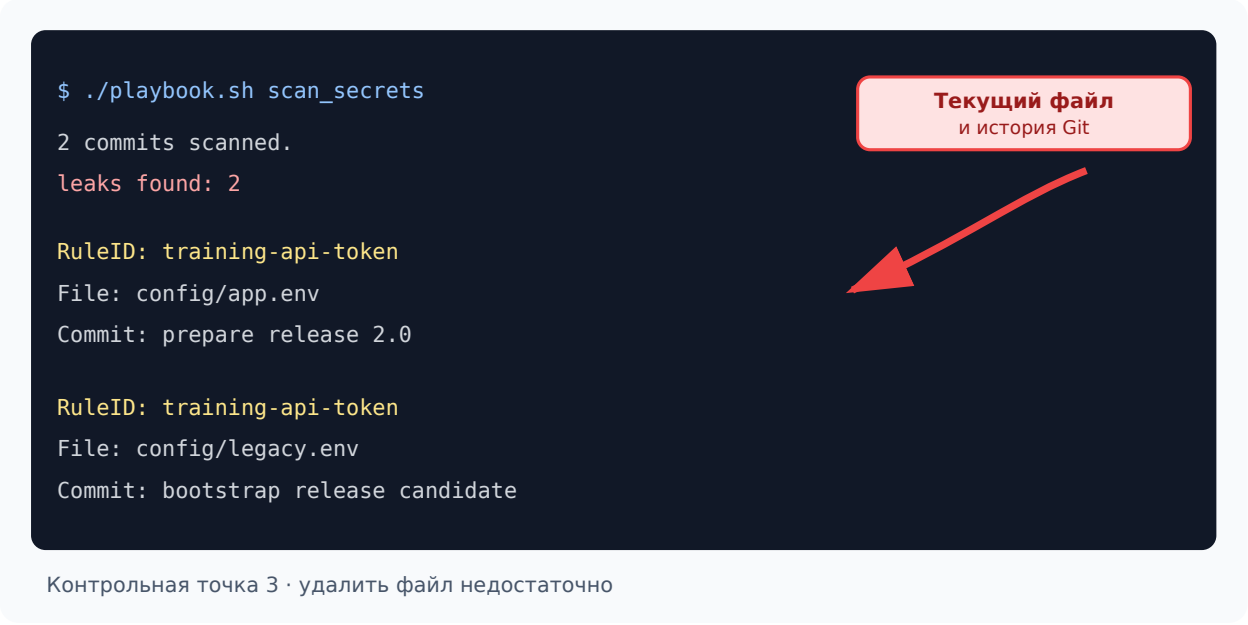


Рисунок 3 — Удаление файла из рабочей копии не удаляет секрет из коммита

5. Образ, SBOM и зависимости — 12 минут

В `build_release()` запишите:

```
docker build -t practice-2-release:student .
```

В `make_sbom()`:

```
syft practice-2-release:student \
  -o cyclonedx-json=/work/reports/sbom.cdx.json
```

В `scan_dependencies()`:

```
grype sbom:/work/reports/sbom.cdx.json \
  -o json --file /work/reports/grype.json || true
grype sbom:/work/reports/sbom.cdx.json --only-fixed || true
```

Последовательно выполните:

```
./playbook.sh build_release
./playbook.sh make_sbom
./playbook.sh scan_dependencies
```

`sbom.cdx.json` — перечень компонентов, а `grype.json` — сопоставление этих компонентов с базой уязвимостей. Подсчитайте их:

```
jq '.components | length' /work/reports/sbom.cdx.json
jq '.matches | length' /work/reports/grype.json
jq -r '.matches[].vulnerability.severity' /work/reports/grype.json | sort | uniq -c
```

Вторая команда Grype с `--only-fixed` показывает только уязвимости, для которых в базе есть исправленная версия пакета. Для Debian 12 таких может не оказаться, и на экране будет `No vulnerabilities found`: это не значит, что уязвимостей нет, полный перечень находится в `grype.json`. Число CVE изменяется при обновлении базы; решение принимается по тяжести, наличию исправленной версии и достижимости компонента. Обратите внимание, сколько сборочных пакетов попало в runtime-образ.

6. Контейнер — 9 минут

В `scan_container()` запишите:

```
trivy config --skip-check-update --format json \
  --output /work/reports/trivy.json . || true
trivy config --skip-check-update --severity HIGH,CRITICAL . || true
```

Выполните `./playbook.sh scan_container`. Исходный Dockerfile не задаёт непривилегированного пользователя и устанавливает лишние рекомендуемые пакеты. В `compose.yaml` дополнительно видны `privileged: true` и запись исходного каталога внутрь контейнера; эти риски исправляются даже если конкретная версия Trivy не показывает их отдельными findings.

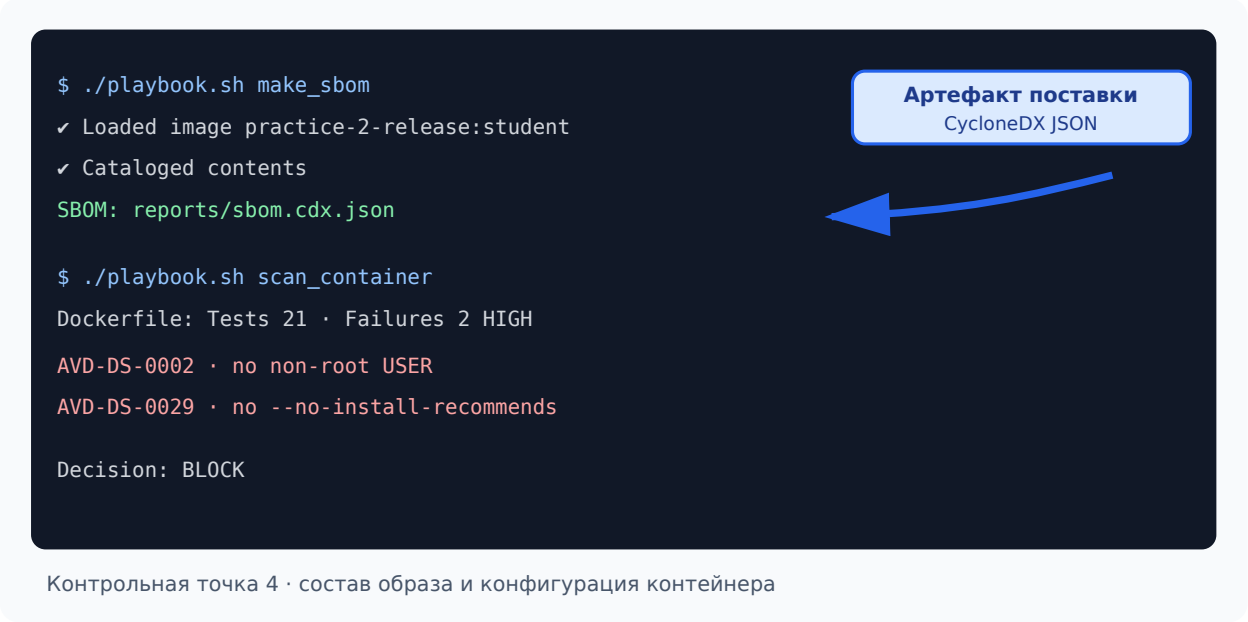


Рисунок 4 — SBOM отвечает «что поставляем», Trivy — «как это запускаем»

Перед исправлением сохраните отчёты: повторный Gate перезапишет их, а исходные числа понадобятся для сравнения «до» и «после».

```
cp -r /work/reports /work/reports-before
```

7. Исправление — 6 минут

Перейдите к исходникам: `cd /work/repository`. Для редактирования используйте установленный в образе `nano`, например `nano src/session.c`.

7.1. Единая точка освобождения

В `/work/repository/src/session.c` замените функцию `handle_session`:

```
int handle_session(int reject_request) {
    int status = 0;
    char *session = allocate_session();
    if (session == NULL) {
        return 2;
    }

    if (reject_request) {
        fprintf(stderr, "request rejected: %s\n", session);
        status = 1;
        goto cleanup;
    }

    puts("session closed");

cleanup:
    free(session);
    return status;
}
```

7.2. Проверка индекса

В конец `src/session.c` добавьте проверку границ в функцию `session_quota`:

```
int session_quota(int slot) {
    if (slot < 0 || slot >= 4) {
        return -1;
    }
    return quota[slot];
}
```

Функция `main` уже обрабатывает значение `-1` как `invalid slot`. Теперь запуск с `--slot 9` печатает `invalid slot`, а ASAN молчит.

7.3. Контейнер

Переделайте Dockerfile в multi-stage: компилятор остаётся на стадии `build`, в runtime копируется только бинарный файл. Пакеты ставятся с `--no-install-recommends`, процесс работает от системного пользователя `user`, добавлен `HEALTHCHECK`. Замените содержимое `Dockerfile`:

```
FROM debian:12-slim AS build

RUN apt-get update \
    && apt-get install -y --no-install-recommends build-essential cmake \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY CMakeLists.txt ./
COPY include ./include
COPY src ./src
RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
    && cmake --build build \
    && ctest --test-dir build

FROM debian:12-slim
RUN groupadd --system app \
    && useradd --system --gid app --create-home user
WORKDIR /app
COPY --from=build --chown=user:app /src/build/release-agent ./release-agent
USER user
HEALTHCHECK --interval=30s --timeout=3s CMD ["/app/release-agent"]
ENTRYPOINT ["/app/release-agent"]
```

В `compose.yaml` удалите `privileged` и bind mount и ограничьте права контейнера:

```
services:
  release-agent:
    build: .
    image: practice-2-release:student
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    tmpfs:
      - /tmp:size=16m,noexec,nosuid
```

7.4. Секреты и история Git

Этот шаг выполняется последним, после правок кода и контейнера. Удалите `config/app.env`, создайте `config/app.env.example` без значения токена и зафиксируйте **все** исправления одним коммитом. Затем, после учебной ротации, очистите оба файла из истории:

```
rm config/app.env
printf 'PAYMENTS_TOKEN=\nLOG_LEVEL=info\n' > config/app.env.example
git add -A
git commit -m "fix release security findings"
FILTER_BRANCH_SQUELCH_WARNING=1 git filter-branch -f \
  --index-filter 'git rm -q --cached --ignore-unmatch config/legacy.env config/
app.env' \
  --prune-empty -- --all
rm -rf .git/refs/original
git reflog expire --expire=now --all
git gc --prune=now --quiet
```

Команда `git filter-branch` работает только с чистым рабочим каталогом. Если она вывела `Cannot rewrite branches: You have unstaged changes`, значит, часть правок не зафиксирована: выполните `git add -A`, `git commit` и повторите блок очистки. Убедитесь, что токенов в истории не осталось — следующая команда не должна ничего вывести:

```
git log --all -S TRAINING_TOKEN_ --oneline
```

В производственной системе сначала отзывают и заменяют настоящий секрет, затем согласуют переписывание опубликованной истории с командой.

8. Повторный Security Gate — 3 минуты

В `verify_release()` запишите вызовы уже заполненных функций:

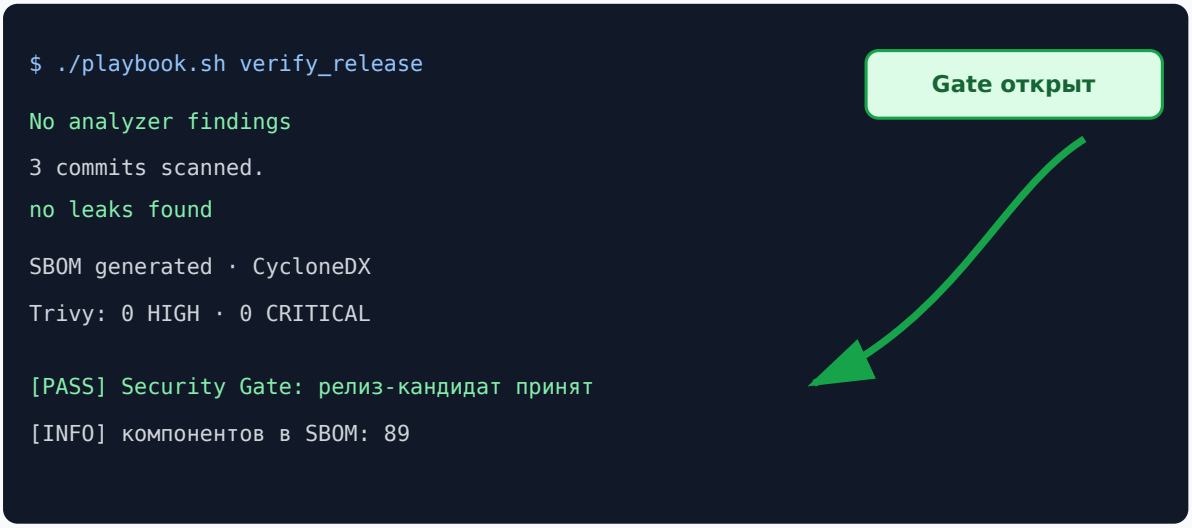
```
scan_sast
scan_dast
scan_secrets
build_release
make_sbom
scan_dependencies
scan_container
python3 /opt/practice/verify.py
```

Функции вызываются напрямую: playbook в начале переходит в `/work/repository`, и вызов `./playbook.sh` изнутри функции не найдёт файл. Вернитесь в `/work` и запустите проверку:

```
cd /work
./playbook.sh verify_release
```

Итоговая строка:

```
[PASS] Security Gate: релиз-кандидат принят
```



Контрольная точка 5 · повторная проверка после исправлений

Рисунок 5 — Gate открывается только по результатам повторного анализа